Technology Stack

iMovies MERN Project • Phase-wise Documentation

Project: iMovies MERN **Document:** Technology Stack

1. Stack summary

Layer	Technology	Purpose
Frontend	React	Component-based user interface.
Frontend tooling	Vite	Development server and production build.
Styling	CSS	Layout, theme and responsive presentation.
Backend runtime	Node.js	JavaScript runtime for API server.
Backend framework	Express.js	REST endpoint routing and middleware.
Database	MongoDB	Document database for persistent application data.
ODM	Mongoose	Schemas, models and database operations.
Authentication	JWT	Token-based authentication for protected routes.
Password security	bcrypt	Password hashing and verification.
Version control	Git	Track project history and changes.
Repository hosting	GitHub	Remote code repository and collaboration.

2. Architecture fit

React/Vite provides the movie catalogue, login/register, booking and admin interfaces. Express exposes API routes. Mongoose models map application records to MongoDB. JWT middleware protects selected endpoints; bcrypt supports password hashing.

3. Development prerequisites

- Install a supported Node.js version and npm.
- Install or configure MongoDB and ensure it is running.
- Install backend dependencies from backend/package.json.
- Install frontend dependencies from frontend/package.json.
- Configure backend environment variables in a local .env file that is not committed.
- Start backend and frontend in separate terminals and verify the health endpoint.

4. Environment configuration

Example local configuration keys may include MONGO_URI, JWT_SECRET, ADMIN_USERNAME and ADMIN_PASSWORD, depending on server.js. Use unique local values, keep the .env file private, and provide a safe .env.example with placeholder values only if needed. Never publish real secrets.

5. Repository hygiene

- Keep node_modules out of Git; regenerate them with npm install.

- Keep frontend/dist out of Git if it is generated during build.
- Keep .env and other secret files out of Git.
- Commit package.json and package-lock.json so dependencies can be installed reproducibly.
- Review GitHub contents and recent commits before submission.

6. Build and verification

Frontend verification: run npm run build from the frontend directory and fix any errors. Backend verification: start the server, confirm MongoDB connects, and check the health endpoint. Functional verification: execute the UAT cases and record actual results.

7. Deployment considerations

A deployed version needs a hosted frontend, a backend service, a reachable MongoDB deployment, production environment variables, HTTPS, configured CORS, and a production API URL. Localhost URLs will not work for remote visitors.

8. Version note

Specific installed package versions should be taken from the project's package.json and package-lock.json at submission time. This document describes the project stack, not a guarantee that every library is the latest available version.

Prepared for academic project documentation. Verify implementation-specific details and fill in actual test evidence before final submission.